

SGT UNIVERSITY

School of Engineering and Technology

Department of CSE

DATABASE MANAGEMENT SYSTEMS LABORATORY

Experiment 6

Nested Queries

Field	Details
Student	Harshit Khemani
Roll No.	241302081
Programme	B.Tech CSE (AI/ML)
Section	Section - C
Lab Date	30 September 2026
Dataset	employee (16 rows; same seed as Experiment 5)
Methods	Subqueries, IN, NOT IN, EXISTS, correlated subqueries

1. Aim

To create the same employee table used in Experiment 5, insert the same 16 rows, and answer the lab questions with nested queries: a scalar subquery, the highest salary in each department, the second-highest salary, IN / NOT IN, EXISTS, a correlated department average, and the department with the greatest total salary.

2. Theory

2.1 Scalar subqueries

A subquery is a SELECT inside another statement. A scalar subquery returns one value and can be compared with > or =. Question 3 compares each salary with the single overall average, 56125.0. That inner query does not mention the outer row, so it is uncorrelated and can be computed once.

2.2 IN and NOT IN

IN (subquery) keeps an outer row when its value appears in the inner result. Question 6 keeps employees whose department is one of the departments that contain a Professor (CSE, Mechanical, ECE). NOT IN is the complement: question 7 keeps Civil, the only department with no Professor. If the subquery could return NULL, NOT IN would be unknown for every row and the result would be empty. Here department is NOT NULL, so that trap does not apply.

2.3 EXISTS and correlated subqueries

A correlated subquery mentions a column from the outer query, so it is re-evaluated for each outer row. EXISTS is true when the inner query returns at least one row; the selected columns do not matter, which is why question 8 selects the constant 1. It lists everyone who has a colleague in the same department on a higher salary — everyone except the highest-paid person in each department. Question 4 uses the same correlation to keep the row whose salary equals the department maximum. Question 9 compares each salary with the average of that employee's own department, which is a different number from the overall average used in question 3.

2.4 Second-highest salary

Question 5 asks, word for word: "Find the 2nd highest salary of the employees." The nested query is SELECT MAX(salary) FROM employee WHERE salary < (SELECT MAX(salary) FROM employee). The inner MAX is the top salary (92000). The outer MAX is the largest salary strictly below that value, which is the next distinct salary. SQLite returns one row: 90000. The same pattern still means second-highest distinct salary if two people share the top salary.

2.5 A nested aggregate

Question 10 asks which department has the greatest total salary. The inner query groups by department and the middle query takes MAX of those totals. The outer HAVING keeps the department whose SUM(salary) equals that maximum. CSE's total is 296000, which is greater than ECE (235000), Mechanical (232000) and Civil (135000).

3. Schema

One table, employee, holds six attributes. It is the same table and the same 16 rows as Experiment 5, created again here so this script runs on its own. sr_no is the employee number (primary key). salary is a positive monthly amount in rupees.

Attribute	Type	Constraint	Role
sr_no	INTEGER	PRIMARY KEY	Employee number
employee_name	TEXT	NOT NULL	Employee name
job	TEXT	NOT NULL	Job title
department	TEXT	NOT NULL	Department name
salary	NUMERIC(10,2)	NOT NULL, CHECK > 0	Monthly salary (Rs)
city	TEXT	NOT NULL	Work city

4. Questions

Ten lab questions. Questions 1 and 2 build the table. Question 3 compares each salary with one overall average. Question 4 finds the highest salary in each department. Question 5 is: Find the 2nd highest salary of the employees. Questions 6 and 7 use IN and NOT IN. Question 8 uses EXISTS. Question 9 is a correlated department average. Question 10 nests an aggregate to find the department with the greatest total salary.

- Create a table employee with the following attributes:
 - sr_no
 - employee_name
 - job
 - department
 - salary
 - city
- Insert 16 rows in the above mentioned table.
Same 16 rows as Experiment 5. Civil has no Professor, so NOT IN is non-empty. The highest salary (92000) and the second-highest salary (90000) are each earned by one employee. Neha Gupta (ECE, 58000) is above the company average and below the ECE average.
- Display the employee name, department and salary of employees who earn more than the average salary, using a nested query.
- Display the employee with the highest salary in each department using a subquery.
- Find the 2nd highest salary of the employees.
- Display employees who work in a department that has a Professor, using IN and a subquery.
- Display employees who do not work in a department that has a Professor, using NOT IN and a subquery.
department is NOT NULL, so the subquery cannot return NULL. A NULL in a NOT IN list would make the predicate unknown for every row.
- Display employees for whom there exists a higher-paid colleague in the same department, using EXISTS.
- Display employees who earn more than the average salary of their own department, using a correlated subquery.
The comparison is strict. Rahul Das earns exactly the Civil average (45000), so this question does not list him. Neha Gupta is above the company average and below the ECE average, so she is in question 3 and not in this question.
- Display the department whose total salary is the greatest, using a nested query.

5. Procedure

Drop employee if it already exists so the script can be re-run. Create the table, insert the 16 rows, then run each query in order. Results that return more than one row are ordered so the same script prints the same rows on every run. The SQL for each question:

Q. No.	SQL solution
1	<pre>CREATE TABLE employee (sr_no INTEGER PRIMARY KEY, employee_name TEXT NOT NULL, job TEXT NOT NULL, department TEXT NOT NULL, salary NUMERIC(10, 2) NOT NULL CHECK (salary > 0), city TEXT NOT NULL);</pre>
2	<pre>INSERT INTO employee (sr_no, employee_name, job, department, salary, city) VALUES (1, 'Rajesh Kumar', 'Professor', 'CSE', 92000.00, 'Delhi'), (2, 'Ananya Sharma', 'Associate Professor', 'CSE', 78000.00, 'Bangalore'), (3, 'Fatima Khan', 'Assistant Professor', 'CSE', 56000.00, 'Bangalore'), (4, 'Karan Iyer', 'Lab Instructor', 'CSE', 42000.00, 'Mumbai'), (5, 'Dev Patel', 'Teaching Assistant', 'CSE', 28000.00, 'Ahmedabad'), (6, 'Vikram Singh', 'Professor', 'Mechanical', 88000.00, 'Hyderabad'), (7, 'Rohan Mehta', 'Associate Professor', 'Mechanical', 64000.00, 'Mumbai'), (8, 'Arjun Patel', 'Assistant Professor', 'Mechanical', 48000.00, 'Pune'), (9, 'Mohit Jain', 'Lab Instructor', 'Mechanical', 32000.00, 'Jaipur'), (10, 'Priya Nair', 'Professor', 'ECE', 90000.00, 'Chennai'), (11, 'Neha Gupta', 'Associate Professor', 'ECE', 58000.00, 'Delhi'), (12, 'Meera Joshi', 'Assistant Professor', 'ECE', 51000.00, 'Chennai'), (13, 'Kavya Menon', 'Lab Instructor', 'ECE', 36000.00, 'Kochi'), (14, 'Sneha Reddy', 'Associate Professor', 'Civil', 60000.00, 'Hyderabad'), (15, 'Rahul Das', 'Assistant Professor', 'Civil', 45000.00, 'Kolkata'), (16, 'Tushar Rao', 'Lab Instructor', 'Civil', 30000.00, 'Nagpur');</pre>
3	<pre>SELECT employee_name, department, salary FROM employee WHERE salary > (SELECT AVG(salary) FROM employee) ORDER BY salary DESC;</pre>
4	<pre>SELECT e.employee_name, e.department, e.salary FROM employee AS e WHERE e.salary = (SELECT MAX(c.salary) FROM employee AS c WHERE c.department = e.department) ORDER BY e.department;</pre>
5	<pre>SELECT MAX(salary) FROM employee WHERE salary < (SELECT MAX(salary) FROM employee);</pre>
6	<pre>SELECT employee_name, job, department, salary FROM employee WHERE department IN (SELECT department FROM employee WHERE job = 'Professor') ORDER BY department, employee_name;</pre>
7	<pre>SELECT employee_name, job, department, salary FROM employee WHERE department NOT IN (SELECT department FROM employee WHERE job = 'Professor') ORDER BY department, employee_name;</pre>

Q. No.	SQL solution
8	<pre> SELECT e.employee_name, e.department, e.salary FROM employee AS e WHERE EXISTS (SELECT 1 FROM employee AS c WHERE c.department = e.department AND c.salary > e.salary) ORDER BY e.department, e.salary DESC; </pre>
9	<pre> SELECT e.employee_name, e.department, e.salary FROM employee AS e WHERE e.salary > (SELECT AVG(d.salary) FROM employee AS d WHERE d.department = e.department) ORDER BY e.department, e.salary DESC; </pre>
10	<pre> SELECT department, SUM(salary) AS total_salary FROM employee GROUP BY department HAVING SUM(salary) = (SELECT MAX(total_salary) FROM (SELECT SUM(salary) AS total_salary FROM employee GROUP BY department) AS dept_totals); </pre>

6. Source Code

SQLite script used in the lab compiler (30-09-2026-2/employee.sql). For the class SQL Server, run employee.sqlserver.sql. The folder name ends in -2 because this is the second experiment dated 30 September 2026.

```

-- =====
-- DBMS Lab · 30-09-2026 · Experiment 6
-- Employee table: CREATE, seed 16 rows, then nested queries
-- Scalar subquery, IN, NOT IN, EXISTS, correlated subquery
-- Folder 30-09-2026-2: second experiment on 30 Sep 2026.
-- SQLite (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run employee.sqlserver.sql instead.
-- Aggregate functions are Experiment 5 (30-09-2026).
--
-- Lab questions
-- 1. Create a table employee with the following attributes:
--   sr no, employee name, job, department, salary, city
-- 2. Insert 16 rows in the above mentioned table
-- 3. Display employees who earn more than the average salary
-- 4. Display the employee with the highest salary in each
--   department using a subquery
-- 5. Find the 2nd highest salary of the employees.
-- 6. Display employees who work in a department that has a
--   Professor, using IN
-- 7. Display employees who do not work in a department that
--   has a Professor, using NOT IN
-- 8. Display employees for whom there EXISTS a higher-paid
--   colleague in the same department
-- 9. Display employees who earn more than the average salary
--   of their own department (correlated subquery)
-- 10. Display the department whose total salary is the greatest
--     using a nested query
-- =====

DROP TABLE IF EXISTS employee;

-- -----
-- 1. Create a table employee with the following attributes:
--   sr no, employee name, job, department, salary, city
-- -----
CREATE TABLE employee (
  sr_no          INTEGER PRIMARY KEY,
  employee_name  TEXT NOT NULL,
  job            TEXT NOT NULL,
  department     TEXT NOT NULL,
  salary         NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
  city          TEXT NOT NULL
);
-- -----

```

```

-- 2. Insert 16 rows in the above mentioned table
--   * same seed as Experiment 5 so the two labs compare
--   * Civil has no Professor, so NOT IN is non-empty
--   * the highest salary (92000) and the second-highest (90000)
--   * are each earned by exactly one employee
--   * Neha Gupta (ECE, 58000) is above the company average
--   * and below the ECE average
-----
INSERT INTO employee
(sr_no, employee_name, job, department, salary, city)
VALUES
(1, 'Rajesh Kumar', 'Professor', 'CSE', 92000.00, 'Delhi'),
(2, 'Ananya Sharma', 'Associate Professor', 'CSE', 78000.00, 'Bangalore'),
(3, 'Fatima Khan', 'Assistant Professor', 'CSE', 56000.00, 'Bangalore'),
(4, 'Karan Iyer', 'Lab Instructor', 'CSE', 42000.00, 'Mumbai'),
(5, 'Dev Patel', 'Teaching Assistant', 'CSE', 28000.00, 'Ahmedabad'),
(6, 'Vikram Singh', 'Professor', 'Mechanical', 88000.00, 'Hyderabad'),
(7, 'Rohan Mehta', 'Associate Professor', 'Mechanical', 64000.00, 'Mumbai'),
(8, 'Arjun Patel', 'Assistant Professor', 'Mechanical', 48000.00, 'Pune'),
(9, 'Mohit Jain', 'Lab Instructor', 'Mechanical', 32000.00, 'Jaipur'),
(10, 'Priya Nair', 'Professor', 'ECE', 90000.00, 'Chennai'),
(11, 'Neha Gupta', 'Associate Professor', 'ECE', 58000.00, 'Delhi'),
(12, 'Meera Joshi', 'Assistant Professor', 'ECE', 51000.00, 'Chennai'),
(13, 'Kavya Menon', 'Lab Instructor', 'ECE', 36000.00, 'Kochi'),
(14, 'Sneha Reddy', 'Associate Professor', 'Civil', 60000.00, 'Hyderabad'),
(15, 'Rahul Das', 'Assistant Professor', 'Civil', 45000.00, 'Kolkata'),
(16, 'Tushar Rao', 'Lab Instructor', 'Civil', 30000.00, 'Nagpur');
-----

-- 3. Display the employee name, department and salary of
--   employees who earn more than the average salary,
--   using a nested query
-----
-- Employees earning more than the average salary
SELECT employee_name, department, salary
FROM employee
WHERE salary > (SELECT AVG(salary) FROM employee)
ORDER BY salary DESC;
-----

-- 4. Display the employee with the highest salary in each
--   department using a subquery
-----
-- Highest salary in each department
SELECT e.employee_name, e.department, e.salary
FROM employee AS e
WHERE e.salary = (
    SELECT MAX(c.salary)
    FROM employee AS c
    WHERE c.department = e.department
)
ORDER BY e.department;
-----

-- 5. Find the 2nd highest salary of the employees.
-----
-- Find the 2nd highest salary of the employees.
SELECT MAX(salary)
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
-----

-- 6. Display employees who work in a department that has a
--   Professor, using IN and a subquery
-----
-- Employees in a department that has a Professor
SELECT employee_name, job, department, salary
FROM employee
WHERE department IN (
    SELECT department
    FROM employee
    WHERE job = 'Professor'
)
ORDER BY department, employee_name;
-----

-- 7. Display employees who do not work in a department that
--   has a Professor, using NOT IN and a subquery
-----
-- Employees not in a department that has a Professor
SELECT employee_name, job, department, salary
FROM employee
WHERE department NOT IN (
    SELECT department
    FROM employee
    WHERE job = 'Professor'
)
ORDER BY department, employee_name;
-----

-- 8. Display employees for whom there EXISTS a higher-paid
--   colleague in the same department (correlated subquery)

```

```

-----
-- Employees with a higher-paid colleague
SELECT e.employee_name, e.department, e.salary
FROM employee AS e
WHERE EXISTS (
    SELECT 1
    FROM employee AS c
    WHERE c.department = e.department
    AND c.salary > e.salary
)
ORDER BY e.department, e.salary DESC;

-----
-- 9. Display employees who earn more than the average salary
-- of their own department (correlated subquery)
-----
-- Employees above their department average
SELECT e.employee_name, e.department, e.salary
FROM employee AS e
WHERE e.salary > (
    SELECT AVG(d.salary)
    FROM employee AS d
    WHERE d.department = e.department
)
ORDER BY e.department, e.salary DESC;

-----
-- 10. Display the department whose total salary is the
-- greatest, using a nested query
-----
-- Department with the greatest total salary
SELECT department,
    SUM(salary) AS total_salary
FROM employee
GROUP BY department
HAVING SUM(salary) = (
    SELECT MAX(total_salary)
    FROM (
        SELECT SUM(salary) AS total_salary
        FROM employee
        GROUP BY department
    ) AS dept_totals
);

```

7. Output

The script was executed in SQLite in memory. CREATE prints a status line and INSERT prints the row count. Each SELECT prints the result set that SQLite returned. Whole-number salaries print as integers.

```

SQL> DROP TABLE IF EXISTS employee;
-- OK

SQL> CREATE TABLE employee (
    sr_no          INTEGER PRIMARY KEY,
    employee_name  TEXT NOT NULL,
    job            TEXT NOT NULL,
    department     TEXT NOT NULL,
    salary         NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
    city           TEXT NOT NULL
);
-- OK

SQL> INSERT INTO employee
(sr_no, employee_name, job, department, salary, city)
VALUES
(1, 'Rajesh Kumar', 'Professor', 'CSE', 92000.00, 'Delhi'),
(2, 'Ananya Sharma', 'Associate Professor', 'CSE', 78000.00, 'Bangalore'),
(3, 'Fatima Khan', 'Assistant Professor', 'CSE', 56000.00, 'Bangalore'),
(4, 'Karan Iyer', 'Lab Instructor', 'CSE', 42000.00, 'Mumbai'),
(5, 'Dev Patel', 'Teaching Assistant', 'CSE', 28000.00, 'Ahmedabad'),
(6, 'Vikram Singh', 'Professor', 'Mechanical', 88000.00, 'Hyderabad'),
(7, 'Rohan Mehta', 'Associate Professor', 'Mechanical', 64000.00, 'Mumbai'),
(8, 'Arjun Patel', 'Assistant Professor', 'Mechanical', 48000.00, 'Pune'),
(9, 'Mohit Jain', 'Lab Instructor', 'Mechanical', 32000.00, 'Jaipur'),
(10, 'Priya Nair', 'Professor', 'ECE', 90000.00, 'Chennai'),
(11, 'Neha Gupta', 'Associate Professor', 'ECE', 58000.00, 'Delhi'),
(12, 'Meera Joshi', 'Assistant Professor', 'ECE', 51000.00, 'Chennai'),
(13, 'Kavya Menon', 'Lab Instructor', 'ECE', 36000.00, 'Kochi'),
(14, 'Sneha Reddy', 'Associate Professor', 'Civil', 60000.00, 'Hyderabad'),
(15, 'Rahul Das', 'Assistant Professor', 'Civil', 45000.00, 'Kolkata'),
(16, 'Tushar Rao', 'Lab Instructor', 'Civil', 30000.00, 'Nagpur');
-- 16 row(s) inserted

SQL> SELECT employee_name, department, salary
FROM employee
WHERE salary > (SELECT AVG(salary) FROM employee)
ORDER BY salary DESC;
employee_name  department  salary
-----
Rajesh Kumar   CSE         92000
Priya Nair     ECE         90000
Vikram Singh   Mechanical  88000
Ananya Sharma  CSE         78000
Rohan Mehta    Mechanical  64000
Sneha Reddy    Civil        60000

```

Neha Gupta ECE 58000

```
SQL> SELECT e.employee_name, e.department, e.salary
FROM employee AS e
WHERE e.salary = (
    SELECT MAX(c.salary)
    FROM employee AS c
    WHERE c.department = e.department
)
ORDER BY e.department;
employee_name department salary
-----
Rajesh Kumar   CSE          92000
Sneha Reddy    Civil         60000
Priya Nair     ECE           90000
Vikram Singh   Mechanical    88000
```

```
SQL> SELECT MAX(salary)
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
MAX(salary)
-----
90000
```

```
SQL> SELECT employee_name, job, department, salary
FROM employee
WHERE department IN (
    SELECT department
    FROM employee
    WHERE job = 'Professor'
)
ORDER BY department, employee_name;
employee_name job department salary
-----
Ananya Sharma Associate Professor CSE 78000
Dev Patel Teaching Assistant CSE 28000
Fatima Khan Assistant Professor CSE 56000
Karan Iyer Lab Instructor CSE 42000
Rajesh Kumar Professor CSE 92000
Kavya Menon Lab Instructor ECE 36000
Meera Joshi Assistant Professor ECE 51000
Neha Gupta Associate Professor ECE 58000
Priya Nair Professor ECE 90000
Arjun Patel Assistant Professor Mechanical 48000
Mohit Jain Lab Instructor Mechanical 32000
Rohan Mehta Associate Professor Mechanical 64000
Vikram Singh Professor Mechanical 88000
```

```
SQL> SELECT employee_name, job, department, salary
FROM employee
WHERE department NOT IN (
    SELECT department
    FROM employee
    WHERE job = 'Professor'
)
ORDER BY department, employee_name;
employee_name job department salary
-----
Rahul Das Assistant Professor Civil 45000
Sneha Reddy Associate Professor Civil 60000
Tushar Rao Lab Instructor Civil 30000
```

```
SQL> SELECT e.employee_name, e.department, e.salary
FROM employee AS e
WHERE EXISTS (
    SELECT 1
    FROM employee AS c
    WHERE c.department = e.department
    AND c.salary > e.salary
)
ORDER BY e.department, e.salary DESC;
employee_name department salary
-----
Ananya Sharma CSE 78000
Fatima Khan CSE 56000
Karan Iyer CSE 42000
Dev Patel CSE 28000
Rahul Das Civil 45000
Tushar Rao Civil 30000
Neha Gupta ECE 58000
Meera Joshi ECE 51000
Kavya Menon ECE 36000
Rohan Mehta Mechanical 64000
Arjun Patel Mechanical 48000
Mohit Jain Mechanical 32000
```

```
SQL> SELECT e.employee_name, e.department, e.salary
FROM employee AS e
WHERE e.salary > (
    SELECT AVG(d.salary)
    FROM employee AS d
    WHERE d.department = e.department
)
ORDER BY e.department, e.salary DESC;
employee_name department salary
-----
Rajesh Kumar CSE 92000
Ananya Sharma CSE 78000
Sneha Reddy Civil 60000
Priya Nair ECE 90000
Vikram Singh Mechanical 88000
Rohan Mehta Mechanical 64000
```

```
SQL> SELECT department,
SUM(salary) AS total_salary
```

```

FROM employee
GROUP BY department
HAVING SUM(salary) = (
    SELECT MAX(total_salary)
    FROM (
        SELECT SUM(salary) AS total_salary
        FROM employee
        GROUP BY department
    ) AS dept_totals
);

```

department	total_salary
CSE	296000

8. Results

Sixteen employees load successfully. Every question returns a non-empty result on this seed. The figures below are the row counts and values from the SQLite run in section 7.

Q. No.	Rows	What the result shows
1	—	Table employee created with 6 attributes
2	16	16 rows inserted, the same seed as Experiment 5
3	7	Names earning more than the overall average 56125.0
4	4	Rajesh Kumar, Sneha Reddy, Priya Nair, Vikram Singh
5	1	2nd highest salary = 90000
6	13	IN: employees in CSE, Mechanical or ECE (a Professor exists)
7	3	NOT IN: Rahul Das, Sneha Reddy and Tushar Rao (Civil)
8	12	EXISTS: everyone except the 4 department-wise top salaries
9	6	Salary greater than that department's own average
10	1	CSE, total salary 296000, the greatest department total

Question 3 returns seven employees above 56125.0, including Neha Gupta at 58000. Question 9 returns six employees above their own department average and does not include her: the ECE average is 58750.0. Fatima Khan (56000) is below both the company average and the CSE average (59200.0). Rahul Das earns exactly the Civil average (45000), so the strict comparison in question 9 excludes him, and 45000 is also below the company average. Question 8 returns 12 rows: 16 employees minus the four department maxima that question 4 lists. Question 5 returns one value, 90000, the largest salary below 92000.

9. Conclusion

A nested query uses one result as the input of another. A scalar subquery supplies a single comparison value. IN and NOT IN test membership. EXISTS tests whether any inner row matches. When the inner query refers to the outer row it is correlated, so each employee can be compared with their own department rather than with the whole company. The same nesting also answers “which group has the greatest total?” by taking MAX of a grouped SUM. Aggregate functions, which compute those averages and totals, are Experiment 5.